

Dyna-GRPO: World-Model-Augmented Rollouts and Counterfactual Credit Assignment for Multi-Tool Agentic RL

A one-month research roadmap — problem formulation, methodology, experiments, and deliverables

Target venue: NeurIPS (main track) • Base model: Qwen3-4B • Benchmarks: AIME 2024/2025, GPQA-Diamond, LiveCodeBench-v6 • Tools: code interpreter, web search, calculator

1. Unified Problem Statement

Problem. Scaling reinforcement learning for multi-tool reasoning agents is bottlenecked by two compounding inefficiencies. First, every on-policy rollout triggers real external tool executions (code sandboxes, web APIs, calculators), making a single gradient update orders of magnitude more expensive than standard reasoning RL. Second, group-relative policy optimization (GRPO) and its variants propagate a single scalar reward uniformly across every token and every tool call in a trajectory, so the optimizer cannot tell which tool choice in a multi-step chain actually caused success or failure — leading to miscalibrated *tool-selection bias* (e.g., over-invoking code when search was needed, or vice versa).

Proposal. We introduce **Dyna-GRPO**, a unified framework in which a set of lightweight, calibrated *tool world-models* are trained to predict tool outputs (and an uncertainty score) from (query, tool, arguments) triples. These predictors enable two mechanisms on top of GRPO: (i) **Dyna-style mixed rollouts** that substitute real tool calls with simulated ones whenever the world-model's predictive uncertainty is below a learned threshold, cutting wall-clock rollout cost substantially; and (ii) **counterfactual tool credit assignment**, in which we re-simulate trajectories with alternative tool choices at each decision point and use the difference in simulated outcomes to form per-tool-call advantage corrections. Together, these produce a training signal that is both cheaper to compute and more causally faithful to each tool's contribution.

Contributions. (1) A calibrated tool-simulation architecture with selective-execution gating; (2) a Dyna-style GRPO variant with bias-corrected mixed-real/imagined advantage estimators; (3) a counterfactual per-tool credit assignment scheme with provable reduction in gradient variance under standard assumptions; (4) an empirical study on four reasoning benchmarks showing training-time speedup and measurable reduction in tool-selection bias.

1.1 Why this is NeurIPS-worthy

- Unification.** A single mechanism — learned tool world-models — solves two superficially distinct problems (rollout cost + credit assignment), mirroring how Dyna unified planning and learning in classical RL.
- Generality.** The method is tool-agnostic and GRPO-agnostic; it composes with DAPO, GSPO, RLOO, and future group-based variants.
- Measurable axes.** Speedup, pass@1, and a novel *tool-selection bias* metric give the paper clean quantitative hooks.
- Timely.** Multi-tool agentic RL (ReTool, ToRL, SkyRL, rStar-Agent) is exploding; no prior work combines model-based simulation with counterfactual credit assignment in this setting.

1.2 Formal setup

A trajectory $\tau = (s_0, a_0, o_0, s_1, a_1, o_1, \dots, s_T)$ consists of states s_t (prompt + context), actions a_t (a text token or a structured tool call (k, args) with $k \in \{\text{code}, \text{search}, \text{calc}, \emptyset\}$), and observations o_t (tool output or empty). The policy $\pi_\theta(a_t | s_t)$ is the LLM. Reward $R(\tau)$ is task-level (final-answer correctness plus format). Standard GRPO computes a group-normalized advantage $\hat{A}(\tau) = (R(\tau) - \mu_G) / \sigma_G$ and applies it uniformly to every token. Our framework replaces this with

$$\hat{A}_t(\tau) = \hat{A}(\tau) + \lambda \cdot \Delta_t^{\text{CF}}(\tau) \quad \text{where} \quad \Delta_t^{\text{CF}}(\tau) = R(\tau) - \mathbb{E}_{a'_t \sim \hat{\pi}}[\hat{R}(\tau_{\leq t}, a'_t, \psi)]$$

Here ψ is the tool world-model, $\tau_{\leq t}$ is the trajectory prefix, and $\hat{R}$ is the simulated reward under a counterfactual tool action a'_t . The first term preserves the GRPO signal; the second adds a per-decision counterfactual correction. A calibrated gating function $g(s_t, a_t) \in \{\text{real}, \text{simulated}\}$ decides whether to invoke the real tool or the world-model during rollout.

2. Technical Approach

2.1 Tool world-models (predictors)

- Architecture.** One small predictor per tool ($\leq 0.5\text{B}$ params, initialized from Qwen3-0.6B or a distilled T5-small). Input: $(\text{tool_name}, \text{query_context}, \text{args})$; output: $(\text{predicted_output_text}, \text{uncertainty_score } u \in [0, 1])$.
- Training data.** Offline corpus of 50–200K real $(\text{args} \rightarrow \text{output})$ pairs collected by running the base Qwen3-4B on a held-out prompt set. Cached outputs become supervised targets.
- Calibration.** Uncertainty head is trained with a proper scoring rule (Brier / focal loss) against an exact-match signal. Post-hoc temperature scaling on a validation split ensures $\text{ECE} < 0.05$.
- Gating.** Simulate when $u < \tau_k$ (tool-specific threshold chosen per a target simulation-fidelity floor, e.g., 92% match with real tool on a probe set).

2.2 Dyna-style mixed rollouts

- During each GRPO step we sample G rollouts per prompt. For each tool call, the gate decides real vs. simulated.
- Bias control.** A fraction η of rollouts (e.g., 20%) is forced to use only real tools, giving an unbiased anchor group. The final advantage combines grouped normalized rewards with an importance-style correction to account for imagined transitions.
- Expected speedup.** Real tool latency (especially code exec / web search) dominates rollout time; even 50% simulation replacement yields $\sim 1.8\text{--}2.5\times$ wall-clock speedup in preliminary estimation.

2.3 Counterfactual per-tool credit assignment

- At every tool-call step t , enumerate K counterfactual tool actions a'_t (alternative tool or alternative args sampled from the policy's top- K).
- Roll out *only the suffix* from s_t using the world-model to get $\hat{r}$. This is cheap — no real tool calls — so we can afford dense CF estimation.
- The per-step correction Δ_t^{CF} is applied only to tokens inside the tool-call span, isolating credit to the decision that caused the divergence.
- **Variance guarantee.** Using a learned baseline $\hat{V}(s_t)$, we show (under standard assumptions) that the CF-corrected gradient has variance $\leq$ the GRPO gradient plus a term that vanishes as world-model fidelity $\rightarrow 1$.

2.4 Tool-selection bias metric (novel)

We define **Tool-Selection KL (TS-KL)**: on a held-out oracle-labeled set where each query has a ground-truth optimal tool k^* , measure $\text{KL}(\pi_\theta(k \mid s) \parallel \delta_{\{k^*\}})$ averaged across the set. Lower is better. We report TS-KL at initialization, after GRPO, and after Dyna-GRPO.

3. One-Month Week-by-Week Plan

Week	Milestone	Concrete deliverables
Week 1 Infra	Baselines, tool harness, and data collection	(a) Reproduce vanilla GRPO on Qwen3-4B with a single tool (code interpreter) on AIME-2024; confirm numbers match published results within ± 1 pp. (b) Build the three-tool harness: Python sandbox, web-search API (serper / tavily), numeric calculator; unified <code>tool_call</code> JSON schema. (c) Collect 100K (tool, args, output) pairs by running Qwen3-4B on a 5K-prompt mix from NuminaMath, GPQA-train-like, and SWE-bench-lite queries. Cache everything.
Week 2 Model	Tool world-models + calibration	(a) Train three per-tool predictors (distilled small LLM or Qwen3-0.6B LoRA). (b) Add uncertainty head + temperature scaling; target ECE < 0.05 on val set. (c) Measure predictor fidelity: exact-match / BLEU / numeric-tolerance against real tool outputs. (d) Choose gating thresholds τ_k such that simulated-call fidelity $\geq 92\%$. <i>Stretch: shared-backbone multi-tool predictor as ablation.</i>
Week 3 Algo	Dyna-GRPO + counterfactual credit assignment	(a) Implement mixed-rollout sampler (real vs. simulated gate, anchor-group fraction η). (b) Implement counterfactual re-simulation: top- K alternative tool actions, suffix rollout via world-model, compute Δ_t^{CF} . (c) Plug the corrected advantage into the GRPO loss; verify gradients are finite and loss decreases on a tiny sanity subset. (d) Short training run (2-3 steps) to debug numerics, memory, and distributed scheduling.
Week 4 Eval	Full training, evaluation, ablations, writing	(a) Full training run (Qwen3-4B, 3 seeds) on AIME, GPQA-Diamond, LiveCodeBench-v6 mixed data. (b) Evaluate pass@1 on all four benchmarks. (c) Ablations: no world-model (= GRPO), no CF correction, varying η and τ , shared vs. per-tool predictor, sensitivity to predictor fidelity. (d) Report wall-clock speedup and TS-KL reduction. (e) Draft sections 1-5 of the paper; produce all figures.

Risk buffer. Reserve the final 2-3 days of Week 4 for re-runs, seed averaging, and failure-case analysis. If full training stalls on GPU quota, fall back to Qwen3-1.7B for ablations and run Qwen3-4B only for the headline numbers.

4. Detailed Daily Plan (First Two Weeks)

Day	Task
Day 1	Fork a GRPO codebase (verl / OpenRLHF / TRL-GRPO). Run vanilla GRPO end-to-end with the code-interpreter tool on a toy AIME subset; confirm checkpointing and logging.
Day 2	Build tool abstraction layer. All three tools expose <code>call(args) → (output, latency_ms, error)</code> . Add a caching layer (sqlite) so identical args never re-execute.
Day 3	Dataset curation: 5K queries spanning math, coding, and knowledge. Run Qwen3-4B with ReAct-style prompting to collect $\sim 100K$ tool-call traces.
Day 4	Reproduce a published Qwen3-4B + GRPO baseline on AIME-2024; record pass@1 and rollout latency breakdown (tool time vs. generation time).
Day 5	Finalize splits: train / val / calibration / probe. Document in a data card.
Day 6-7	<i>Slack / writing.</i> Draft paper introduction and method skeleton while baselines run.
Day 8	Train per-tool predictor for the <i>calculator</i> (easiest — highly deterministic). Validate exact-match on held-out args.
Day 9	Train the <i>code interpreter</i> predictor (medium — structured but variable). Evaluate on held-out problems.

Day 10	Train the <i>web search</i> predictor (hardest — open-ended output). Consider constraining to top-3 titles + snippets to reduce output variance.
Day 11	Add uncertainty heads; fit temperature scaling on val split. Compute ECE; iterate until ECE < 0.05 per tool.
Day 12	Fidelity probe: on 2K held-out traces, measure real-vs-simulated agreement. Pick τ_k per tool.
Day 13	Integration test: replace real tool calls with gated simulated calls inside a rollout loop; measure latency reduction on 100 rollouts.
Day 14	Write the "Tool World-Models" methods subsection while results are fresh.

5. Experimental Protocol

5.1 Training configuration

- **Base model.** Qwen3-4B-Instruct (with tool-use prompting). LoRA rank 64 on attention + MLP for RL updates to keep memory manageable; full-parameter as a sanity follow-up.
- **GRPO setup.** Group size $G = 8$; KL coefficient 0.001; clip ratio 0.2; max 4 tool calls per trajectory; max 4096 generation tokens.
- **Hardware.** 4× A100-80G (or 8× A6000); predictors served via vLLM alongside the actor.
- **Seeds.** 3 seeds for headline numbers, 1 seed for ablations.

5.2 Benchmarks and metrics

- **AIME 2024 / 2025.** pass@1 over 32 samples (standard), math competition problems requiring calculator + code.
- **GPQA-Diamond.** pass@1, knowledge-intensive — stresses web search.
- **LiveCodeBench-v6.** pass@1, tests code interpreter use.
- **Primary metrics.** Pass@1 (quality), wall-clock training time to reach a target reward (efficiency), TS-KL (tool-selection calibration).
- **Secondary.** Per-tool invocation frequency, counterfactual advantage magnitude distribution, predictor-fidelity sensitivity curves.

5.3 Baselines

- Qwen3-4B zero-shot with tool use (no RL).
- Qwen3-4B + vanilla GRPO (primary baseline).
- Qwen3-4B + DAPO (strong recent baseline).
- Qwen3-4B + GRPO with uniform token-level advantage (to isolate the CF-correction benefit).
- Qwen3-4B + Dyna-GRPO without CF correction (to isolate the world-model benefit).
- Qwen3-4B + Dyna-GRPO (full method).

5.4 Ablations

- **Predictor size.** 0.5B vs. 1B vs. shared 1.5B backbone.
- **Anchor-group fraction** $\eta \in \{0, 0.1, 0.2, 0.4, 1.0\}$. $\eta = 1.0$ recovers vanilla GRPO.
- **Gating threshold** τ — trade speedup vs. fidelity.
- **Counterfactual K** — number of alternative tool actions re-simulated per step.
- **Fidelity stress test.** Artificially corrupt predictor outputs and measure downstream pass@1; confirms robustness of gating.

6. Expected Results and Success Criteria

- **Speedup.** $\geq 1.8\times$ wall-clock training speedup at matched final reward vs. vanilla GRPO.
- **Quality.** Pass@1 within ± 0.5 pp of GRPO on AIME/GPQA/LCB (quality preserved); $\geq +1.5$ pp improvement on the mixed-tool benchmark when CF correction is enabled.
- **Calibration.** $\geq 25\%$ reduction in TS-KL versus vanilla GRPO.
- **Scaling pointer.** Evidence that gains grow with more tools (from 2 $\rightarrow$ 3 tools), motivating larger follow-up at scale.

7. Risks and Mitigations

Risk	Likelihood / Impact	Mitigation
Predictors too inaccurate for web search	Medium / High	Restrict search-predictor target to structured top-3 snippets; fall back to real calls when u high.
CF advantages inject bias $\rightarrow$ pass@1 drops	Medium / High	Anchor-group fraction η ; clipped CF advantage; ablate λ .
Compute budget exceeded	Medium / Medium	Drop to Qwen3-1.7B for ablations; reduce seed count to 2 on ablations.
GRPO training instability	Low / High	Use DAPO-style clipping and low-variance KL; inherit verl's tested implementation.
World-model drift during training	Medium / Medium	Periodically (every 200 steps) refresh predictors on recent real-tool traces.

8. Deliverables at End of Month

1. **Codebase.** Clean, reproducible implementation of Dyna-GRPO, predictors, CF credit assignment, and the tool harness.
2. **Model checkpoints.** Qwen3-4B Dyna-GRPO trained on all three tools, plus baseline checkpoints.
3. **Paper draft.** ~8-page NeurIPS-format draft with figures, tables, and ablations ready for polishing.
4. **Supplementary.** Data card, predictor fidelity report, and a 2-page technical appendix with the variance-reduction proof sketch.
5. **Internal talk.** 20-minute results readout with a demo of simulated vs. real tool calls on a live prompt.

9. Positioning Against Related Work

Agentic RL with tools. ReTool, ToRL, SkyRL-Agent, rStar-Agent show that RL on tool-using LLMs works but all perform real tool calls per rollout; none uses learned simulation or per-tool credit. **GRPO variants.** DAPO and GSPO improve group-based optimization but still assign credit at trajectory or token level, not *per tool call*. **Model-based RL.** Classical Dyna, MuZero, and DreamerV3 train world-models for full environment dynamics; our setting is narrower (tool I/O) and therefore much more tractable to model. **Credit assignment in LLMs.** Process reward models (PRMs) and step-DPO assign step-level credit via learned reward models; we instead derive credit counterfactually from a simulator, sidestepping the need for annotated step labels.

The combination — *narrow learned world-models + counterfactual re-simulation + group-relative RL* — is, to our knowledge, new.

10. Preparatory Checklist (Before Day 1)

- Spin up verl or OpenRLHF with Qwen3-4B, verify vLLM rollout + training loop on a single node.
- Provision keys/quotas: web-search API, a sandboxed code executor, sufficient GPU hours for ~4 weeks.
- Read and summarize: ReTool, ToRL, DAPO, DreamerV3's world-model losses, and one counterfactual-RL paper (e.g., counterfactual multi-agent policy gradients).
- Pre-register evaluation protocol (benchmarks, metrics, seeds) to prevent post-hoc fishing.